UNIVERSITATEA OVIDIUS DIN CONSTANȚA
FACULTATEA DE MATEMATICĂ ȘI INFORMATICĂ
COMPUTER SCIENCE

LUCRARE DE LICENȚĂ
Raport de progres

Network Intrusion Detection System
Signature-based and Machine-Learning-based
with Built-in Forensics

Coordonator ştiinţific:
Prof. Dorin Iordache

Absolvent:
Bora Cristian-Vasile

Constanţa, 2026

Abstract

This report describes the current state of Ovidius NIDS, a Network Intrusion Detection System (NIDS) that combines classical signature-based rule matching with a supervised / unsupervised machine-learning detector and exposes both as a self-contained Security Information and Event Management (SIEM) console. The system captures network packets in real time (via Scapy [2] or a deterministic synthetic traffic generator), evaluates each packet against a YAML-seeded rule base, scores it with a scikit-learn [11] classifier (Random Forest [3] or Isolation Forest [8]) and persists, for every triggered alert, a ring-buffered packet snapshot to disk so that the analyst can replay the events surrounding the incident—a built-in forensics layer that distinguishes this project from the typical academic IDS prototype.

The current implementation reaches Milestone 3 of the planned roadmap: the backend (FastAPI [13], async SQLAlchemy, JWT authentication, WebSocket live streams) and the React dashboard (Vite [18], TailwindCSS, Recharts) are end-to-end functional, the hybrid detection pipeline produces alerts on synthetic attack traffic, and a one-click cross-platform launcher abstracts the setup steps. This document summarises the motivation, theoretical background, architecture, implementation choices and the work remaining before the final thesis defense.

Keywords: Intrusion Detection, Network Security, Machine Learning, Random Forest, Isolation Forest, SIEM, Forensics, FastAPI, React.

Contents

Abstract **1**

1 Introduction **4**
1.1 Context and motivation 4
1.2 Problem statement 4
1.3 Contribution of this stage 5

2 Theoretical background **5**
2.1 Network Intrusion Detection Systems 5
2.2 Signature-based detection 6
2.3 Machine learning for intrusion detection 6
2.4 Hybrid detection 7
2.5 Forensics in a SIEM 7

3 Related work **8**
3.1 Production-grade NIDS 8
3.2 Machine-learning IDS 8
3.3 SIEM and forensics platforms 9
3.4 Positioning of this work 9

4 System architecture **9**
4.1 High-level overview 9
4.2 Component breakdown 10
4.2.1 Capture 10
4.2.2 Dispatcher 10
4.2.3 Signature engine 10
4.2.4 ML detector 11
4.2.5 Forensics layer 11
4.2.6 REST + WebSocket API 11
4.2.7 Frontend 11
4.3 Data model 11
4.4 One-click launcher 12

5 Implementation details **12**
5.1 Technology stack 12
5.2 Packet model 12
5.3 Detection pipeline core 13
5.4 Authentication and authorisation 14

5.5 Forensic file format 14
5.6 Resilience and recent stabilisation work 14
5.7 Source-tree organisation 15

6 Current state of the project **16**
6.1 Functional milestones reached 16
6.2 Quantitative observations 16
6.3 Known limitations 17
6.4 Repository state 17

7 Remaining work and outlook **18**
7.1 Before the final thesis defense 18
7.2 Stretch goals 18
7.3 Risks and mitigations 19
7.4 Conclusion of this progress report 19

References **19**

1 Introduction

1.1 Context and motivation

Network attacks have moved from a niche concern to a mainstream business risk: modern enterprises rely on continuously inspecting their traffic for malicious patterns, and the discipline that formalises that inspection is *Network Intrusion Detection*. Two complementary schools of thought dominate the field. **Signature-based** systems — Snort, Suricata and Zeek being the canonical references [9, 10, 14] — pattern-match every packet (or every flow) against a hand-curated rule base, producing high-precision alerts but inevitably missing attacks for which no rule has yet been written [6]. **Anomaly-based** (or machine-learning-based) systems instead learn a model of "normal" behaviour and flag deviations [4]; they cope better with previously-unseen attacks but pay for that flexibility with a higher false-positive rate and the well-known interpretability deficit famously critiqued by Sommer and Paxson [16].

Real-world Security Operations Centres rarely deploy one school in isolation. They run a signature engine for the cheap, high-confidence detections *and* a machine-learning detector as a second opinion, and they need a way to investigate any alert in detail—often by replaying the precise packets that led up to it. That last requirement (*forensics*) is the most under-served in open-source academic work: the typical research prototype evaluates a classifier on a static dataset and stops there, leaving the analyst experience entirely out of scope.

1.2 Problem statement

The project addresses three concrete questions:

1. Can a single self-hostable application combine signature and machine-learning detection in a way where the two engines reinforce each other (*hybrid* detection) rather than running side-by-side as silos?

2. Can such a system persist enough context per alert that an analyst can perform meaningful triage without an external packet sniffer? Specifically: which packets immediately preceded the alert, what was their payload, what was the inter-arrival timing?

3. Can the whole thing be packaged so that a non-developer (a coordinator, a defense committee, an evaluator at the *Faculty of Mathematics and Computer Science*) can launch it with a double-click and see results within seconds, without Python, Node or virtual-environment knowledge?

1.3 Contribution of this stage

At the time of this progress report, the project contributes:

- A modular detection pipeline (`Dispatcher` – see Section 4) where signature and ML verdicts are merged into three explicit alert categories: *signature-only*, *ml-only*, and *hybrid*.
- A YAML-seeded, hot-reloadable rule store editable from the UI, avoiding the restart-to-edit penalty common in Snort/Suricata deployments.
- A per-alert ring-buffered packet capture written as JSON Lines on disk, downloadable from the UI through an authenticated endpoint.
- A React + Tailwind SIEM console (login, dashboard, alerts, rules, forensics, ML models, settings) with live WebSocket updates for both packet rate and alert stream.
- A one-click cross-platform launcher (`run.py` / `run.bat` / `run.sh`) that creates a virtual environment, installs dependencies, starts both services, opens the browser, and handles graceful shutdown—designed specifically for the *open-and-demonstrate* scenario typical of a thesis defense.

The remainder of this document is organised as follows. Section 2 reviews the necessary background. Section 3 situates the project relative to existing tools and research. Section 4 describes the system design. Section 5 discusses the implementation details. Section 6 reports what is functional today, with quantitative observations gathered during development. Section 7 closes with the items remaining before the final defense.

2 Theoretical background

2.1 Network Intrusion Detection Systems

A Network Intrusion Detection System inspects network traffic — either passively, by tapping into a SPAN port or virtual interface, or actively, as part of a forwarding plane (NIPS) — and emits an alert when the observed packets match a model of malicious activity [4, 6]. NIDS taxonomies typically split detection strategies along two orthogonal axes:

Detection paradigm. *Misuse* (a.k.a. *signature-based*) detection encodes known attack patterns; *anomaly-based* detection encodes normal behaviour and reports deviations.

Granularity. *Packet-level* detection inspects each frame independently; *flow-level* detection aggregates packets into bidirectional 5-tuple flows before scoring.

Ovidius NIDS operates packet-level for the signature engine (rules can match any combination of L3/L4 fields, payload regular expressions, or content substrings) and per-packet for the current ML detector, with flow-level features (inter-arrival time, src-rate) maintained as streaming state. This trade-off favours real-time responsiveness over the larger context windows used by flow-based systems such as Zeek [10].

2.2 Signature-based detection

The dominant open-source signature systems use a domain-specific language for rules [9, 14]. A typical Snort rule is structured as a header (action, protocol, addresses, ports) followed by a comma-separated set of options that constrain payload content, session state, flow direction and so on. Ovidius NIDS adopts a simpler, JSON-based match specification (Listing 1) which is expressive enough for the categories of attack the project targets (scans, brute-force, command injection, reverse shell, simple flood) without requiring users to learn a third-party DSL.

Listing 1: Example signature rule, stored both in YAML and in the database

```
- name: "SSH brute force"
  category: "BruteForce"
  severity: "high"
  match:
    protocol: tcp
    dst_port: 22
    flags: "S"
    threshold:
      count: 10
      window_sec: 30
      group_by: src_ip
  action: alert
```

2.3 Machine learning for intrusion detection

The two algorithms exposed by Ovidius NIDS were chosen for their complementary semantics rather than for state-of-the-art performance:

Random Forest [3] is a supervised ensemble of decision trees trained on labelled $(features, y)$ pairs. It produces calibrated probability estimates, exposes per-feature importance (used to populate the dashboard bar chart), and is robust to noisy or irrelevant features. This makes it a natural fit for the "second opinion" role described in the introduction.

Isolation Forest [8] is an unsupervised anomaly detector that recursively partitions the feature space with random splits; points that require few splits to be isolated are

scored as anomalies. The algorithm does not require labelled attack data, which matters in practice because attack traffic is far rarer than benign traffic and is hard to label reliably [16].

The feature vector currently used by both algorithms is [src_port, dst_port, $length$, $proto_id$, $flag_mo$ — six numeric features computed per packet, with the inter-arrival time maintained as streaming state keyed by source IP. This intentionally small vector keeps inference cheap (sub-millisecond per packet on a laptop) and avoids the dataset-leakage pitfalls described by Sommer and Paxson [16], where features that encode the collection methodology rather than the underlying behaviour cause models to over-fit.

2.4 Hybrid detection

When both engines run concurrently, three alert categories are emitted:

- **signature-only** — the rule engine matched, the ML model disagreed (label = benign).
- **ml-only** — no rule matched, the ML model scored the packet above a configurable threshold (currently 0.8).
- **hybrid** — both engines agreed. Hybrid alerts carry the highest confidence and are surfaced first in the dashboard triage queue.

This explicit labelling makes the analyst aware of *why* an alert fired and avoids the black-box critique levelled at pure ML IDS [16].

2.5 Forensics in a SIEM

The Security Information and Event Management acronym covers the broader class of systems that aggregate, correlate and present security events [1]. In a production SIEM, forensics is typically delegated to an external packet capture appliance. Ovidius NIDS takes the opposite approach: every alert triggers a write of the last $N = 50$ packets seen by the dispatcher (kept in a circular buffer of size 200) to a per-alert JSON Lines file. The file is associated with the alert via a foreign-key relationship in the SQLite database and is served back to the UI through an authenticated FastAPI endpoint. This brings the most common triage workflow — "show me what happened just before this alert fired" — into the application itself.

3 Related work

3.1 Production-grade NIDS

Snort [14] is the original lightweight signature IDS and remains the de-facto reference. Its strengths are maturity, a vast rule ecosystem (Emerging Threats, Talos), and a permissive license. Its main shortcoming for the use case targeted here is that it is a console-only daemon: integrating Snort into an analyst workflow requires either a commercial console (Snort Cloud) or a third party such as SecurityOnion.

Suricata [9] re-engineered Snort for multi-threaded operation and added protocol-aware parsing. It remains rule-driven and, like Snort, ships no built-in dashboard.

Zeek (formerly Bro) [10] occupies a different niche: it logs richly-structured events about every flow and lets operators write detection logic in a scripting language. It is widely used in research but, again, has no embedded analyst UI.

The common gap shared by all three is forensic context: alerts are emitted with timestamp and 5-tuple, but the raw packets that triggered them are typically not retained. Operators bolt on full packet capture (tcpdump, Moloch/Arkime) to fill that gap. Ovidius NIDS treats forensics as a first-class concern of the detection engine itself.

3.2 Machine-learning IDS

The literature on applying machine learning to intrusion detection is vast; recent surveys list hundreds of distinct proposals [4, 7]. Most of this work focuses on benchmark accuracy on synthetic datasets: KDD'99 [17], the cleaned NSL-KDD, and more recent dataset releases such as CICIDS2017 [15].

A persistent critique of this line of research is that benchmark accuracy does not transfer to operational deployment. Sommer and Paxson [16] argue that the closed-world assumption of standard ML evaluation does not hold in networking, where the distribution of benign traffic itself drifts continuously. They recommend (i) anomaly detection rather than misuse detection for the ML component, (ii) operator-friendly explanations of why a model fired, and (iii) keeping the ML engine in an advisory role behind a signature engine. Ovidius NIDS embraces all three recommendations: Isolation Forest is offered as a first-class option, every alert is labelled with its detection method, and hybrid agreement is highlighted prominently in the UI.

3.3 SIEM and forensics platforms

Wazuh, **SecurityOnion** and **Splunk** are fully-fledged SIEM platforms with analyst consoles. They are too large to deploy on a single laptop and they treat detection as an orchestration problem (collecting alerts from external engines) rather than as a unified pipeline. Ovidius NIDS occupies the much smaller niche of *a single-binary teaching/demo SIEM with detection and forensics built in*, intentionally trading scale for clarity.

3.4 Positioning of this work

	Snort	Suricata	Zeek	ML-IDS research	**Ovidius NIDS**
Signature engine	yes	yes	partial	rare	**yes**
ML engine	no	no	no	yes	**yes (RF, IF)**
Hybrid labelling	–	–	–	–	**yes**
Embedded analyst UI	no	no	no	no	**yes**
Per-alert packet capture	no	no	no	no	**yes**
Single-binary deployment	no	no	no	–	**yes (laptop demo)**

Table 1: Positioning of Ovidius NIDS against existing tool categories.

The intention is not to outperform mature production engines on raw throughput or rule count, but to demonstrate, in a single contained application, that the three ingredients — signature, ML and forensics — can be made to cooperate inside a tidy SIEM-style user experience.

4 System architecture

4.1 High-level overview

Ovidius NIDS is a two-tier application: a Python backend that performs capture, detection and persistence; and a single-page React frontend that consumes its REST and WebSocket APIs. The two communicate over HTTP/JSON for queries and over two dedicated WebSocket channels (`/ws/alerts` and `/ws/traffic`) for real-time push. Figure 1 sketches the packet life-cycle.

```
NIC / Simulator → Sniffer → Dispatcher → [ Signature engine + ML detector ]
                → merge → DB (Alert, PcapRecord, TrafficEvent)
                → per-alert JSONL ring on disk + WebSocket broadcast → Browser
```

Figure 1: End-to-end packet flow from capture to UI.

4.2 Component breakdown

4.2.1 Capture

Two implementations share a common asynchronous callback signature:

- `LiveSniffer` uses Scapy [2] on a real interface. It runs on a dedicated thread and posts each parsed `Packet` to the main event loop via `asyncio.run_coroutine_threadsafe`.
- `TrafficSimulator` emits a steady benign baseline of 24–40 packets per second and injects an attack burst every five seconds (Nmap-like scan, SSH brute-force, ICMP flood, SQL injection, command injection, reverse shell). It exists so that the application can be demonstrated on a laptop without the elevated privileges required by raw packet capture, and so that the ML detector always has labelled-ish data to react to.

4.2.2 Dispatcher

The `Dispatcher` is the central coordination point. For every received `Packet` it:

1. Increments the packet counter and the sliding-window packets-per-second meter.
2. Pushes the packet onto a fixed-size ring buffer (`deque` of size 200), which acts as the source for forensic snapshots.
3. Publishes a compact event onto the `traffic_bus` so that any subscribed WebSocket clients (i.e. the dashboard) see live throughput.
4. Optionally persists a `TrafficEvent` record (sampled at 1/5 packets to keep the events table small during demos).
5. Calls the signature engine and the ML detector and merges their verdicts into zero-or-more alert candidates.
6. For each candidate, writes a JSONL ring-tail to disk, creates `Alert` and `PcapRecord` rows, and broadcasts the alert on `alert_bus`.

4.2.3 Signature engine

The engine compiles each enabled DB row into an in-memory `CompiledRule`: a struct of pre-compiled regular expressions, port ranges, flag masks, and (for threshold rules) a sliding-window counter keyed by source IP. Compilation happens once at startup and again whenever the rules table is mutated through the API (hot reload). For every incoming packet, the engine iterates all enabled rules and yields one alert-candidate dictionary per match.

4.2.4 ML detector

The `MLDetector` class wraps both algorithms behind a common `score(pkt)` → `(label, score)` interface. State (current model, feature names, per-source last-seen timestamp) is protected by a mutex so that training can run concurrently with inference. Persistence uses `joblib`; activating a model loads the saved pickle and flips the singleton's active flag.

4.2.5 Forensics layer

When an alert is created, `write_packet_log()` serialises the last 50 entries of the dispatcher's ring buffer as JSON Lines into a file named after the alert ID. The path is stored on the `Alert` row, and a separate `PcapRecord` row mirrors it for table-driven browsing in the UI. Download is served by an authenticated FastAPI endpoint that performs a path check and streams the file with `FileResponse`.

4.2.6 REST + WebSocket API

The HTTP surface is organised by tag in FastAPI: `/api/auth`, `/api/alerts`, `/api/rules`, `/api/dashboard`, `/api/forensics`, `/api/ml`, `/api/health`. All resource routes are protected by `Depends(get_current_user)`, which decodes a JWT and reloads the caller from the database. Two WebSocket endpoints push real-time data: `/ws/alerts` (each new alert as a single message) and `/ws/traffic` (batched traffic samples at ≈ 4 Hz to keep the dashboard chart smooth without overwhelming the network).

4.2.7 Frontend

The dashboard is a Vite [18]-bundled React application styled with TailwindCSS. State is held mostly in React Context (auth, live alerts, live traffic); pages such as `Alerts` and `Rules` fetch via authenticated REST and reconcile against the live WebSocket stream client-side. The visual identity (deep navy with cyan accents) is applied through a Tailwind theme to keep the look consistent across all pages and the new modal components introduced in Section 6.

4.3 Data model

The application uses five SQLAlchemy ORM tables:

User account credentials and role (admin/analyst). Passwords are hashed with bcrypt [12].

Rule one row per signature; the `match` field is a JSON blob whose shape mirrors Listing 1.

Alert the central record. Carries severity, category, detection method (signature/ml/hybrid), ML score, source/destination 5-tuple, status (new/triaged/closed), and the

path to its associated PCAP file.

PcapRecord denormalised view of the forensic file for the UI Forensics table; references `Alert` via foreign key.

TrafficEvent sampled record of traffic for historical analysis.

MLModel persisted metadata for trained models (algorithm, path, metrics, feature importance, active flag).

The schema is created with `Base.metadata.create_all()` on first boot; an Alembic migration suite is planned for the final thesis to support schema evolution without losing data.

4.4 One-click launcher

The launcher (`run.py`) is a deliberate design choice rather than an afterthought. It performs, in order: Python version check (refuses ≥ 3.14 because scientific packages lack Windows wheels for very recent CPython releases, which would force a Visual C++ build); creation of `backend/.venv`; dependency installation against a pinned `requirements.txt`; frontend `npm install`; parallel launch of `uvicorn` and `vite dev`; browser auto-open; supervised waiting on Ctrl+C with cooperative shutdown of both children. The wrappers `run.bat` (Windows) and `run.sh` (macOS/Linux) merely pick the best available Python interpreter and delegate to `run.py`.

5 Implementation details

5.1 Technology stack

The choice of FastAPI was driven by three properties: native asynchronous support (necessary because the packet pipeline is event-loop driven), an automatically generated OpenAPI schema (useful for the defense demonstration), and very low boilerplate for dependency-injected authentication. SQLite was retained for the same reason that motivated the one-click launcher: zero installation footprint.

5.2 Packet model

The internal `Packet` dataclass is intentionally framework-agnostic so that the `LiveSniffer` and the `TrafficSimulator` can feed it interchangeably:

Listing 2: Internal packet representation

```
@dataclass
```

Layer	Choice
Backend framework	FastAPI [13] on Uvicorn
ORM	SQLAlchemy 2 (async) with aiosqlite
Authentication	OAuth2 password flow → JWT signed with HS256
Password hashing	bcrypt [12] (direct calls, no passlib)
Packet capture	Scapy 2.5+ [2]
Machine learning	scikit-learn [11] (RandomForest, IsolationForest)
Numerical	NumPy
Validation	Pydantic v2
Frontend	React 18 + Vite [18]
Styling	TailwindCSS 3
Charts	Recharts
Icons	lucide-react
Test & packaging	pytest, joblib, virtualenv

Table 2: Technology stack as of the current milestone.

```
class Packet:
    ts: datetime # UTC, timezone-aware
    protocol: str # "TCP" | "UDP" | "ICMP" | "OTHER"
    src_ip: str
    src_port: int
    dst_ip: str
    dst_port: int
    length: int # full L2 length
    flags: str # comma-separated TCP flags
    payload: bytes
```

5.3 Detection pipeline core

The heart of the system is the `Dispatcher.handle` coroutine. It shows three concerns kept deliberately separate: metrics, publication, and detection.

Listing 3: Dispatcher entry point (excerpt)

```
async def handle(self, pkt: Packet) -> None:
    self._pkt_count += 1
    self._ring.append(pkt)
    self._tick_pps()

    await self._publish_traffic(pkt)
    await self._maybe_persist_event(pkt)

    alerts = self._evaluate(pkt)
    if alerts:
        await self._persist_and_broadcast(alerts)
```

The merge logic that produces the three alert categories (signature-only, ml-only, hybrid) lives in `_evaluate`; it is the locus of the design choice highlighted in Section 2.

5.4 Authentication and authorisation

Login uses the OAuth2 password flow exposed by FastAPI; tokens are JWTs (HS256) carrying `sub` (username) and an optional role claim, with a default 60-minute expiry. Every protected route declares `Depends(get_current_user)`, which decodes the token, loads the matching `User` row and rejects expired or unknown credentials with HTTP 401. WebSocket endpoints follow the same path but accept the token as a query-string parameter, since browser WebSocket clients cannot set custom request headers [5].

5.5 Forensic file format

Each per-alert capture file is a JSON Lines document where every line encodes one packet (timestamp, 5-tuple, flags, length, base64-encoded payload preview). The format was chosen over PCAP because:

- It is line-delimited, so the dashboard can stream a paginated preview without parsing the whole file.
- It is human-readable, which matters for academic demonstration.
- It avoids the dependency on libpcap-format encoding (Scapy's `wrpcap`) which complicates portability to operating systems without a libpcap install.

5.6 Resilience and recent stabilisation work

During the latest iteration, several rough edges identified in everyday use were addressed:

First-login race. On a cold boot the very first `POST /api/auth/login` could race against the lifespan seeding phase and return HTTP 500. The endpoint now catches the transient database exception, returns a clean 503, and the frontend retries up to three times with linear back-off, eliminating the "mash the button until it works" artefact.

Idempotent shutdown. The launcher's shutdown routine is now guarded by a flag so that the trio of atexit, signal handler and `finally` clause does not run shutdown three times. On Windows this previously required the user to confirm the "Terminate batch job?" prompt twice.

Authenticated download. The Forensics page now downloads files through a `fetch`-based helper that attaches the JWT in the Authorization header and converts the

response to a `Blob` URL; the previous `<a download>` link failed because anchor navigation cannot set custom headers, which produced an opaque "File wasn't available" Chrome error.

Filter correctness. The Alerts page filters (severity, method, status, free-text) now apply to both the historical rows fetched from the API and to the live WebSocket stream; previously the live stream bypassed the filter predicate.

Themed confirmation dialog. Destructive actions (currently *Delete rule*) raise a navy-cyan modal component consistent with the rest of the interface instead of the browser's native pop-up.

ML registry collisions. Re-training with the default model name no longer crashes on the `UNIQUE` constraint; the backend auto-suffixes a free name and the frontend defaults the name per algorithm.

Settings page. The previously read-only Settings tab now offers a change-password form (backed by `POST /api/auth/change-password`), surfaces the current signed-in user and role, and documents the deployment configuration knobs.

5.7 Source-tree organisation

Listing 4: Repository layout (abridged)

```
Licenta_NIDS/
|-- backend/
| |-- app/
| | |-- api/ FastAPI routers
| | |-- core/ config, database, security
| | |-- engine/ sniffer, dispatcher, signatures, ML detector
| | |-- models/ SQLAlchemy ORM
| | |-- schemas/ Pydantic
| | |-- ws/ WebSocket endpoints
| | `-- main.py app factory + lifespan
| |-- data/
| | |-- rules/default_rules.yaml
| | `-- models/ *.joblib
| `-- requirements.txt
|-- frontend/
| |-- src/
| | |-- api/ HTTP + WS client
| | |-- components/ shared UI (modals, badges, sidebar)
| | |-- pages/ Login, Dashboard, Alerts, AlertDetail,
| | | Rules, Forensics, MLModels, Settings
| | `-- store/auth.jsx auth context
| `-- package.json
|-- docs/progress-report/ this document
|-- run.py cross-platform launcher
|-- run.bat Windows wrapper
|-- run.sh Unix wrapper
`-- README.md
```

6 Current state of the project

6.1 Functional milestones reached

1. **Backend skeleton.** FastAPI application with async SQLAlchemy, JWT auth, OpenAPI documentation auto-served at `/docs`.

2. **Signature engine.** YAML seed file, DB-backed editable rules, hot reload from the API, six bundled categories (Recon, BruteForce, Injection, RemoteAccess, DenialOfService, DataExfiltration).

3. **Capture layer.** Both LiveSniffer (Scapy) and the synthetic TrafficSimulator are functional; the simulator currently drives 99% of demonstration usage because raw capture requires elevated privileges.

4. **ML detector.** Random Forest and Isolation Forest can be trained from the UI on a synthetic feature set; metrics, feature importance and the active model are surfaced on the ML Models page.

5. **Forensics.** Per-alert JSONL ring captures are produced, listed and downloadable; the Alert Detail page renders the packet timeline for replay.

6. **Frontend.** Login, Dashboard, Alerts, Alert Detail, Rules, Forensics, ML Models and Settings pages are feature-complete for the current milestone. Live updates via WebSocket cover the dashboard chart and the alert stream.

7. **Deployment.** The one-click launcher works on Windows, macOS and Linux; `run.bat` is a double-click entry point and the auto-installer creates the venv and installs both Python and Node dependencies on first launch.

6.2 Quantitative observations

The figures below are not formal benchmarks; they were collected during regular development on a laptop (Windows 11, Intel i7, 16 GB RAM, Python 3.12, Node 20) and are reported here only as sanity checks for the upcoming evaluation chapter.

The unsupervised Isolation Forest unsurprisingly trails the supervised Random Forest on this synthetic dataset where labels are available; its purpose is to provide a deployment path when labelled attack data is not available, not to win an accuracy contest.

Observation	Value
Steady-state simulator throughput	24–40 packets/s
Burst peak (during an attack injection)	90–120 packets/s
Median signature-engine evaluation time per packet	< 1 ms
Median ML score (Random Forest) per packet	≈ 0.4 ms
Random Forest training time (synthetic, 4000 samples)	≈ 1.0 s
Isolation Forest training time (same dataset)	≈ 0.7 s
Random Forest accuracy on held-out 25% split	0.99
Isolation Forest accuracy on held-out 25% split	0.83
End-to-end alert latency (packet $\rightarrow$ UI)	< 250 ms
Backend cold start (first request served)	1.5–3 s
Frontend cold start (Vite dev server)	2–5 s

Table 3: Indicative measurements during day-to-day usage.

6.3 Known limitations

- Live packet capture has been tested only on Linux; Windows and macOS require manual installation of WinPcap/Npcap or equivalent, which the launcher does not yet automate.
- The ML feature vector is currently per-packet rather than per-flow; flow-aware aggregation is on the roadmap for the final thesis.
- No clustering or correlation engine yet; alerts are surfaced individually. A future "alert grouping by source IP" view is sketched.
- Database migrations rely on `create_all`. An Alembic suite is planned before any production-style deployment.
- The simulator's attack patterns are deterministic and comparatively easy for the ML detector to flag. Evaluation on the CICIDS2017 [15] dataset is planned for the experimental chapter.

6.4 Repository state

At the time of writing, the repository contains approximately:

- 1,900 lines of backend Python (engine + API + models + schemas + tests scaffolding).
- 2,600 lines of frontend JavaScript/JSX across eight pages and the shared component library.
- 480 lines of YAML/JSON for the default rule base and configuration.

- A single-pass-compilable LaTeX folder (this document) with bibliography and section split.

The default credentials in the seeded admin user are `admin/admin`; the password can now be rotated from the Settings page after first login.

7 Remaining work and outlook

The progress documented in this report covers the first three milestones of the planned development cycle. The remaining items, grouped by priority, are listed below.

7.1 Before the final thesis defense

1. **Flow-level features.** Aggregate packets into bidirectional 5-tuple flows and extend the feature vector with duration, packet count, byte count, mean inter-arrival, and protocol-specific fields. This is expected to improve the ML detector's behaviour on multi-packet attack patterns where a single packet is not informative on its own.

2. **Evaluation on a public dataset.** Reproduce the training and evaluation pipeline on the CICIDS2017 [15] dataset and on NSL-KDD [17] to obtain comparable accuracy/precision/recall/F1 figures alongside the synthetic baseline already reported here.

3. **Alembic migrations.** Replace `Base.metadata.create_all()` with a versioned migration suite to support schema evolution without data loss.

4. **Hardening of the live sniffer.** Document, and where possible automate, the Npcap installation on Windows and the capability flags on Linux (`cap_net_raw`).

5. **Test coverage.** Expand the unit-test scaffolding (currently smoke tests on the signature engine and the dispatcher merge logic) towards an integration-test suite that replays a deterministic packet trace and asserts against the produced alerts.

7.2 Stretch goals

1. **Alert correlation.** Group consecutive alerts that share a source IP and category into a single "incident" view, analogous to the SecurityOnion "case" concept.

2. **User-role separation.** Distinguish read-only analyst from admin in the UI; restrict rule editing to admins.

3. **External rule import.** Translate a subset of Snort rules into the project's JSON match specification, providing a bridge to the larger rule ecosystem.

4. **LLM-assisted triage.** A small chat panel on the Alert Detail page that summarises the captured packets in natural language — a separate exploration tracked as a possible appendix chapter.

7.3 Risks and mitigations

Library compatibility on new Python releases. Scientific wheels for Python 3.14 are not yet published on Windows. The launcher already refuses such interpreters and instructs the user to install Python 3.12. The risk is bounded.

Evaluation realism. Synthetic datasets risk over-stating the ML detector's quality [16]. The planned CICIDS2017 evaluation is the mitigation.

Time budget. The flow-level refactor is the largest single item remaining and is on the critical path. A reduced fallback (flow features computed only on a sliding window, no per-flow state in the database) is available if needed.

7.4 Conclusion of this progress report

The project has reached the point where the three engines — signature, machine-learning, and forensics — cooperate inside a single, self-hostable SIEM experience, accessible through a one-click launcher suitable for academic demonstration. The remaining work is concentrated on improving the ML detector's realism, evaluating it against established public datasets, and tightening the production-readiness of the deployment story. The codebase, the live demonstration of the running application, and this document together constitute the *state of the art* of the bachelor's thesis at the date of this report.

References

[1] Idoia Aguirre and Sergio Alonso. Improving the automation of security information management: a collaborative approach. *IEEE Security & Privacy*, 10(1):55–59, 2012. doi: 10.1109/MSP.2011.153.

[2] Philippe Biondi and the Scapy Community. Scapy: Packet manipulation library. https://scapy.net/, 2024. Accessed 2026.

[3] Leo Breiman. Random Forests. *Machine Learning*, 45(1):5–32, 2001. doi: 10.1023/A:1010933404324.

[4] Anna L. Buczak and Erhan Guven. A Survey of Data Mining and Machine Learning Methods for Cyber Security Intrusion Detection. *IEEE Communications Surveys & Tutorials*, 18(2):1153–1176, 2016. doi: 10.1109/COMST.2015.2494502.

[5] Ian Fette and Alexey Melnikov. The WebSocket Protocol. RFC 6455, IETF, 2011.

[6] P. García-Teodoro, J. Díaz-Verdejo, G. Maciá-Fernández, and E. Vázquez. Anomaly-based network intrusion detection: techniques, systems and challenges. *Computers & Security*, 28(1–2):18–28, 2009. doi: 10.1016/j.cose.2008.08.003.

[7] Ansam Khraisat, Iqbal Gondal, Peter Vamplew, and Joarder Kamruzzaman. Survey of intrusion detection systems: techniques, datasets and challenges. *Cybersecurity*, 2 (1):20, 2019. doi: 10.1186/s42400-019-0038-7.

[8] Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. Isolation Forest. In *Proceedings of the 2008 Eighth IEEE International Conference on Data Mining (ICDM)*, pages 413–422. IEEE, 2008. doi: 10.1109/ICDM.2008.17.

[9] Open Information Security Foundation. Suricata – Open Source IDS / IPS / NSM Engine. https://suricata.io/, 2024. Accessed 2026.

[10] Vern Paxson. Bro: a system for detecting network intruders in real-time. *Computer Networks*, 31(23–24):2435–2463, 1999. doi: 10.1016/S1389-1286(99)00112-7.

[11] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[12] Niels Provos and David Mazières. A Future-Adaptable Password Scheme. USENIX Annual Technical Conference, FREENIX Track, 1999.

[13] Sebastián Ramírez. FastAPI – modern, fast (high-performance) web framework for building APIs with Python. https://fastapi.tiangolo.com/, 2024. Accessed 2026.

[14] Martin Roesch. Snort – Lightweight Intrusion Detection for Networks. In *Proceedings of the 13th USENIX Conference on System Administration (LISA '99)*, pages 229–238. USENIX Association, 1999.

[15] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani. Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization. In *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, pages 108–116, 2018. doi: 10.5220/0006639801080116.

[16] Robin Sommer and Vern Paxson. Outside the Closed World: On Using Machine

Learning for Network Intrusion Detection. In *Proceedings of the 2010 IEEE Symposium on Security and Privacy*, pages 305–316. IEEE, 2010. doi: 10.1109/SP.2010.25.

[17] Mahbod Tavallaee, Ebrahim Bagheri, Wei Lu, and Ali A. Ghorbani. A detailed analysis of the KDD CUP 99 data set. In *Proceedings of the 2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA)*, pages 1–6. IEEE, 2009. doi: 10.1109/CISDA.2009.5356528.

[18] Evan You and the Vite contributors. Vite – next-generation frontend tooling. https://vitejs.dev/, 2024. Accessed 2026.